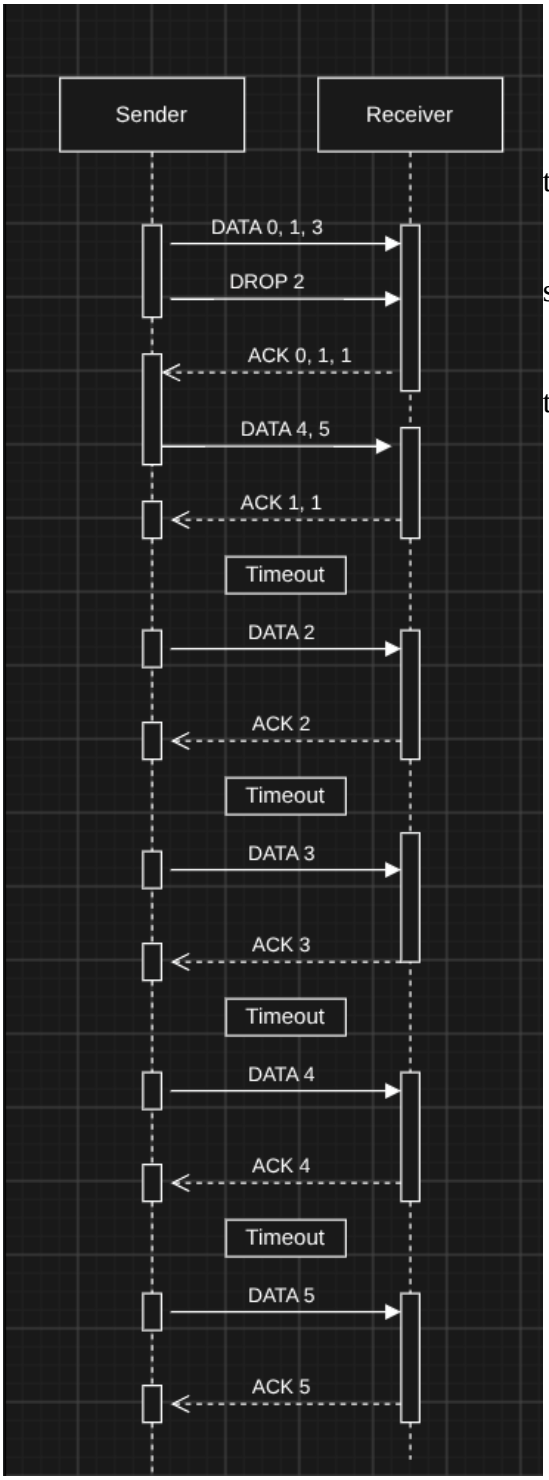
Connor Petri
CS-158A
PA2

Step 1:



1. Data 2-5 should be retransmitted after the first timeout.

2. In scenario 5, it instead waits for additional timeouts to transmit the rest of the window.

3. At the first timeout, self.base = 2 and self.sent_packets = [2, 3, 4, 5]

4. timeout() must retransmit the rest of the packets not just the one.

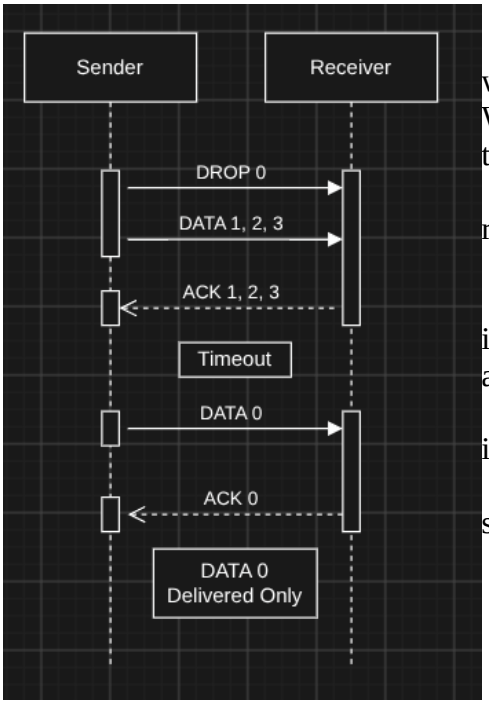
5. Packets 2, 3, 4, 5 all send sequentially

```
[29] t=10 TRANSPORT_RECEIVE (sender) Packet(ACK seq=1 ack=1 data='')
[30] t=15 TIMEOUT (sender) t=15
[31] t=15 TRANSPORT_SEND (sender) Packet(DATA seq=2 ack=2 data='m2') arrives t=17
[32] t=15 TRANSPORT_SEND (sender) Packet(DATA seq=3 ack=3 data='m3') arrives t=17
[33] t=15 TRANSPORT_SEND (sender) Packet(DATA seq=4 ack=4 data='m4') arrives t=17
[34] t=15 TRANSPORT_SEND (sender) Packet(DATA seq=5 ack=5 data='m5') arrives t=17
[35] t=17 TRANSPORT_RECEIVE (receiver) Packet(DATA seq=2 ack=2 data='m2')
[36] t=17 APP_RECEIVE (app) data='m2'
```

Step 2:

1. FAST_RETRANS event happens at t=5
2. Without fast retransmit, seq 0 is resent at t=11.
3. The baseline finished sooner at t=20 rather then t=28 with fast retransmit. In the fast retransmit run, seq 0 arrives at t=7, but the receiver had discarded the out of order packets 1-3. They are not buffered.
4. Fast retransmit works with go-back-n becaude repleated ACKs for the same packet indicate that the next packet is probably missing as well. In selective repeat, ACKs identify individual packetss, so ACKs for later packets confirm receipt rather than signaling a missing base packet.

Step 3:



1. Selective repeat should store each out of order packet within the window in the receiver buffer and ACK each of them individually. When the missing packet(s) are sent, the buffer should be drained to complete the datagram.
2. The trace shows ACKs for 1-3, but the receiver discards them rather than buffering them for later.
3. Each ACK is individual.
4. An ACK should confirm that the individual packet was accepted into the buffer. It does not mean the packet was delivered to the application.
5. Before 0 arrives, self.buffer should contain 1-3. Before the fix, it was empty on each out-of-order arrival.
6. I fixed the out-of-order branch to store packet.data in the self.buffer

```
cpetri@clouds:~/School/networking/assignments/PA2/pa2$ python run_sim.py --protocol sr --scenario 9
Protocol : sr
Scenario : 9 -- SR out-of-order buffering probe
          The 1st DATA packet (seq0) is dropped while seq1..seq3 still arrive and get ACKed individually -- the defining SR
behavior. A correct receiver buffers seq1-3 and delivers everything once seq0 is resent. If out-of-order packets are discarde
d instead of buffered, those messages are gone for good even though the sender was told they arrived: delivered output will b
e missing messages entirely, not just delayed.

[1] t=1 APP_SEND (app) data='m0'
[2] t=1 DROP (sim) Packet(DATA seq=0 ack=0 data='m0') (send #1)
[3] t=1 APP_SEND (app) data='m1'
[4] t=1 TRANSPORT_SEND (sender) Packet(DATA seq=1 ack=1 data='m1') arrives t=3
[5] t=1 APP_SEND (app) data='m2'
[6] t=1 TRANSPORT_SEND (sender) Packet(DATA seq=2 ack=2 data='m2') arrives t=3
[7] t=1 APP_SEND (app) data='m3'
[8] t=1 TRANSPORT_SEND (sender) Packet(DATA seq=3 ack=3 data='m3') arrives t=3
[9] t=3 TRANSPORT_RECEIVE (receiver) Packet(DATA seq=1 ack=1 data='m1')
[10] t=3 TRANSPORT_SEND (receiver) Packet(ACK seq=1 ack=1 data='') arrives t=5
[11] t=3 TRANSPORT_RECEIVE (receiver) Packet(DATA seq=2 ack=2 data='m2')
[12] t=3 TRANSPORT_SEND (receiver) Packet(ACK seq=2 ack=2 data='') arrives t=5
[13] t=3 TRANSPORT_RECEIVE (receiver) Packet(DATA seq=3 ack=3 data='m3')
[14] t=3 TRANSPORT_SEND (receiver) Packet(ACK seq=3 ack=3 data='') arrives t=5
[15] t=5 TRANSPORT_RECEIVE (sender) Packet(ACK seq=1 ack=1 data='')
[16] t=5 TRANSPORT_RECEIVE (sender) Packet(ACK seq=2 ack=2 data='')
[17] t=5 TRANSPORT_RECEIVE (sender) Packet(ACK seq=3 ack=3 data='')
[18] t=11 TIMEOUT (sender) key=0 t=11
[19] t=11 TRANSPORT_SEND (sender) Packet(DATA seq=0 ack=0 data='m0') arrives t=13
[20] t=13 TRANSPORT_RECEIVE (receiver) Packet(DATA seq=0 ack=0 data='m0')
[21] t=13 TRANSPORT_SEND (receiver) Packet(ACK seq=0 ack=0 data='') arrives t=15
[22] t=13 APP_RECEIVE (app) data='m0'
[23] t=13 APP_RECEIVE (app) data='m1'
[24] t=13 APP_RECEIVE (app) data='m2'
[25] t=13 APP_RECEIVE (app) data='m3'
[26] t=15 TRANSPORT_RECEIVE (sender) Packet(ACK seq=0 ack=0 data='')
[27] t=15 DONE (sim) delivered=4 ticks=15

Sent messages     : ['m0', 'm1', 'm2', 'm3']
Delivered messages: ['m0', 'm1', 'm2', 'm3']
Match             : True
Event summary     : {'APP_SEND': 4, 'DROP': 1, 'TRANSPORT_SEND': 8, 'TRANSPORT_RECEIVE': 8, 'TIMEOUT': 1, 'APP_RECEIVE': 4, '
DONE': 1}
```

Step 4:

1. With WINDOW = 8, the sequence number can collide. If 0 remains unACKed and 1-7 are transmitted, then 8 wraps back to 0 before the first packet is ACKed.
2. The receiver would need an additional identifier to distinguish the old 0 from the new 0
3. The rule is to keep WINDOW <= NUM_SEQNUMS / 2 so the sender's window and the reciever's window don't overlap across a wraparound.

Step 5:

1. Go-back-n has 2: FAST_RETRANS at event 19, t=5 and TIMOUT at event 24, t=19. Selective repeat has 1: Timeout at event 18, t=11.
2. Go-back-n discards out-of-order packets and requires retransmission of the entire window starting with the dropped packet. Selective repeat buffers out-of-order packets and only requires the rebroadcasting of dropped packets.
3.Without fast retransmit, the first timeout would be at t=11 as the timer is set at t=1 and has a duration of 10 ticks. Go-back-n would then retransmit 1-3. The run should finish at t=15 after the last of the ACKs come in. This finishes sooner than with FAST_RETRANS.